

Implementation of a simple RISC V processor

Introduction

The focus of this task is on completing an implementation of a lightweight RISC V processor in Verilog and evaluating the implementation with the use of test benches. The lightweight RISC V processor uses a reduced instruction set consisting of only seven instructions:

Two instructions for loading data from the memory into the registers and storing data from the registers to the memory, four arithmetic operations as well as one instruction for conditional branching:

Instruction	Description
ld	Load a value
sd	Store a value
add	Add two values
sub	Subtract two values
and	Bitwise AND operation
or	Bitwise OR operation
beq	Branch if equal

The processor architecture is made up of several components as shown in Fig.1.

The instruction memory loads the instruction indicated by the program counter. It is a read-only memory (ROM) that is initialized with a program made up of 32-bit RISC V instructions.

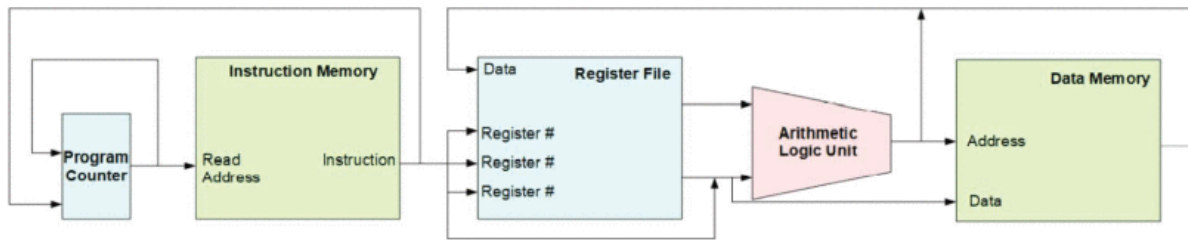


Figure 1: main components of the processor architecture

The register file consists of 32 general-purpose registers. To enable writing a value (`w_data`) to a register, a control signal (`ctrl_reg_w`) is set. The corresponding register is selected using a selection signal (`reg_num_w`). Per clock cycle, the architecture also allows parallel read access to two registers. To determine whether the value to be stored in a register comes from the memory or the ALU, a control signal (`ctrl_mem_to_reg`) is utilized. This signal is set to low when the ALU's output is to be stored in a register.

The ALU is used to perform the logical and arithmetic operations `add`, `sub`, `and`, `or`. Additionally, the `ld`, `sd` and `beq` instructions require calculations as well. For this, the ALU has a zero flag (`zero`) that is set to high if the result of the calculations equals zero. A control signal (`ctrl_ALU_op`) is used to indicate which type of operation is executed on the ALU:

Operations	Ctrl_ALU_op value
<code>ld</code> , <code>sd</code>	00
<code>beq</code>	01
<code>add</code> , <code>sub</code> , <code>and</code> , <code>or</code>	10

Since the `ld`, and `sd` operations require one of the ALU's inputs to be an immediate value, a control signal (`ctrl_ALU_src`) is set to 1 in these cases to select the offset as the second input. For the other operations, two registers are used as inputs.

The data memory is made up of two memory regions, a read-only memory (ROM) segment and a random-access memory (RAM) segment. The lower addresses (i.e. addresses smaller than `DATA_MEMORY_ROM_DEPTH`) are used to access the read-only part of the memory. With the higher addresses, the RAM can be read and written. To read or write to memory the corresponding control signal must be set (`ctrl_mem_w`, `ctrl_mem_r`). For write operation the address also has to be a valid RAM address, i.e. not pointing to the ROM segment of the memory.

All components are controlled by the control unit which takes the instructions' opcode and sets the control signals of the other components accordingly. This includes indicating to the program counter whether the currently executed instruction is a branch instruction (`ctrl_branch`).

Task description

1. Open the task on EDA Playground: <https://www.edaplayground.com/x/k2NM> or get the files from ILIAS.
2. Create a copy of the project.
3. Get an overview over the file structure and the implemented components.
4. Complete the *ALU.sv* file by finishing all the items marked as TODO:
 - a. Implement the logical and arithmetic operations in the ALU (and, or, add, sub).
 - b. Implement the functionality for the zero flag.
5. Complete the data memory by filling in the TODOs in the *data_memory.sv* file
 - a. Reading from ROM and RAM.
 - i. Differentiate for each address whether it points to a date in the ROM or in the RAM.
 - ii. Only read when the correct control signal is set, else return 0.
 - b. Writing to RAM
 - i. Only allow writing when the correct control signal is set.
 - ii. Only allow writing to addresses referencing data in the RAM segment of the memory.
 - c. Resetting RAM entries to zero if reset signal is set.
6. Complete the TODOs in the implementation of the register file (*register_file.sv*)
 - a. Reset registers to zero in case of a reset.
 - b. Write data from the data input to the selected register when the control signal for writing is set.
7. Finish the implementation of state machine in the control module (*control_unit.sv*)
 - a. For each state of the state machine, set the control signals to the correct value.
8. Test your implementation by running a simple program.

Hints

- Start with the implementation of the ALU, data memory and register file. Evaluate the functionality of your implementation of each component with the provided test benches.
- If each of these modules work correctly, proceed to the control module. You can then test the whole design with a test bench (*top_tb.sv*). Since at this point, all other parts of the implementation should work correctly, this can help you to find errors in your control unit implementation.
- The *top_tb.sv* test bench executes a program that calculates the Fibonacci sequence. This means that, if everything is implemented correctly, it should display the sequence: 0, 1, 2, 3, 5, 8, 13, ...
- The *common_pkg* module defines some constants that can be used in the other modules, such as the size of the ROM and RAM